

```
X1 !1 ok1 R1 (stats)1 (dir)1 (fix)1 (reload)1 (wait)1 (pin)1 , com-  
mentstyle=, keywordstyle=, numberstyle=, stringstyle=, basicstyle=, breakatwhitespace=false,  
breaklines=true, captionpos=b, keepspaces=true, numbers=left, numbersep=5pt,  
showspaces=false, showstringspaces=false, showtabs=false, tabsize=4, literate=  
-1 -1 -1 -1 -1 -1 -1 -1 |1  $\times$ 1  $\pm$ /-1  $\sqrt{\text{sqrt}}$ 1
```

ClimateProcessingEngine

A Comprehensive Python Framework for Climate Data
Processing

Version 2.1

Documentation

Amin Fazl Kazemi

amin.fazl.kazemi@example.com

July 24, 2026

Contents

1	Introduction	5
1.1	Overview	5
1.1.1	Key Features	5
1.1.2	Target Audience	5
1.1.3	System Requirements	6
1.1.4	Dependencies	6
2	Architecture Overview	7
2.1	System Architecture	7
2.2	Core Components	7
2.2.1	Configuration Layer (constants.py + config.yaml)	7
2.2.2	I/O Pipeline (io_pipeline/*.py)	7
2.2.3	Numerical Engine (numerical_engine/*.py)	8
2.2.4	Result Pipeline (result_pipeline/*.py)	8
2.2.5	Monitoring (monitoring/*.py)	8
2.2.6	Orchestrator (orchestrator/*.py)	8
2.3	Data Flow	8
3	Installation and Setup	9
3.1	Installation	9
3.1.1	Clone the Repository	9
3.1.2	Create a Virtual Environment	9
3.1.3	Install Dependencies	9
3.2	Configuration	9
3.2.1	Configuration File Structure	9
3.2.2	Understanding Configuration Parameters	11
3.3	Preparing Input Data	11
3.3.1	Input Zarr Structure	11
3.3.2	Calendar File Format	12
4	Core Modules Reference	13
4.1	Configuration Module (constants.py)	13
4.1.1	Overview	13
4.1.2	Key Constants	13
4.1.3	Configuration Functions	14
4.2	Zarr Schema Module (zarr_schema.py)	14
4.2.1	Overview	14
4.2.2	Variable Definitions	14
4.2.3	Key Functions	14

4.3	Calendar Tables Module (calendar_tables.py)	15
4.3.1	Overview	15
4.3.2	Key Functions	15
4.4	Runtime Tables Module (runtime_tables.py)	16
4.4.1	Overview	16
4.4.2	Key Functions	16
4.5	I/O Pipeline Module (io_pipeline/*.py)	17
4.5.1	read_month_files.py	17
4.5.2	assemble_block.py	18
4.5.3	validate_block.py	18
4.6	Numerical Engine Module (numerical_engine/*.py)	18
4.6.1	window_engine.py	18
4.6.2	distributions.py	19
4.6.3	analyze_station.py	21
4.6.4	merge_results.py	22
4.7	Result Pipeline Module (result_pipeline/*.py)	22
4.7.1	validate_result.py	22
4.7.2	write_block.py	23
4.8	Orchestrator Module (orchestrator/*.py)	23
4.8.1	process_block.py	23
4.9	Monitoring Module (monitoring/*.py)	24
4.9.1	benchmark.py	24
4.9.2	checkpoint.py	24
4.9.3	logger.py	25
5	Data Contract	26
5.1	Load Phase Contract	26
5.1.1	Input	26
5.1.2	Output	26
5.2	Window Phase Contract	26
5.2.1	Input	26
5.2.2	Output	26
5.3	Analyze Phase Contract	27
5.3.1	Input	27
5.3.2	Output	27
5.4	Write Phase Contract	27
5.4.1	Input	27
5.4.2	Output	27
5.5	Constants	27
5.6	Error Types	28
6	Mathematical Background	29
6.1	Distributions	29
6.1.1	Normal Distribution	29
6.1.2	Skew-Normal Distribution	29
6.1.3	Bimodal Normal Distribution	29
6.1.4	Pearson Type III Distribution	30
6.2	Model Selection Criteria	30

6.2.1	AIC (Akaike Information Criterion)	30
6.2.2	AICc (Corrected AIC)	30
6.2.3	BIC (Bayesian Information Criterion)	30
6.3	Bimodality Metrics	30
6.3.1	Ashman's D	30
6.3.2	Bimodality Coefficient	31
6.3.3	Overlap Coefficient	31
6.4	Window-Based Estimation	31
7	Execution Flow	32
7.1	Main Function	32
7.2	Block Processing Flow	32
7.3	Recovery Flow	33
8	Input Data Format	34
8.1	Input Zarr Structure	34
8.2	Data Scaling	34
8.3	Missing Data	34
9	Output Format	35
9.1	Zarr Structure	35
9.2	Metadata	36
10	Performance Optimization	37
10.1	Block Size Selection	37
10.2	Parallel Processing	37
10.3	Memory Management	37
10.4	Performance Tips	38
11	Troubleshooting	39
11.1	Common Errors and Solutions	39
11.1.1	File Not Found	39
11.1.2	Memory Error	39
11.1.3	KeyError	39
11.1.4	ImportError	40
11.1.5	IndentationError	40
11.2	Logging	40
11.2.1	Log Levels	40
11.3	Debugging Tips	41
12	Support and Community	42
12.1	Citation	42
12.2	Repository	42
12.3	License	42
12.4	Contact	42
A	Full Variable Definitions	43
B	Configuration File Reference	45
B.1	Complete config.yaml	45

C	Example Usage	47
C.1	Running the Engine	47
C.2	Resuming After Interruption	47
C.3	Starting Fresh	47
C.4	Analyzing Results	48
C.5	Example: Custom Configuration	48
D	Glossary	49

List of Figures

2.1	System Architecture Overview	7
7.1	Block Processing Steps	32

List of Tables

1.1	Minimum System Requirements	6
3.1	Path Configuration Parameters	11
3.2	Time Configuration Parameters	11
3.3	Processing Configuration Parameters	11
4.1	Output Variables per Temperature Variable	14
5.1	Key Constants	27
5.2	Error Types	28
8.1	Input Zarr Variables	34
9.1	Global Attributes	36
A.1	Complete Variable Definitions	43
D.1	Glossary of Terms	49

Listings

1.1	Required Dependencies	6
3.1	Cloning the Repository	9
3.2	Creating Virtual Environment	9
3.3	Installing Dependencies	9
3.4	Example config.yaml	9
3.5	Input Zarr Directory Structure	12
3.6	calendar.txt Format	12
4.1	Constants Defined	13
4.2	Configuration Functions	14
4.3	Zarr Schema Functions	15
4.4	Calendar Functions	15
4.5	Runtime Tables Functions	17
4.6	read_month_files.py	17
4.7	assemble_block.py	18
4.8	validate_block.py	18
4.9	window_engine.py	18
4.10	Normal Distribution Fitting	19
4.11	Skew-Normal Fitting	19
4.12	Bimodal Fitting	20
4.13	Pearson Type III Fitting	20
4.14	fit_all_distributions_numba	20
4.15	analyze_station.py	21
4.16	merge_results.py	22
4.17	validate_result.py	22
4.18	write_block.py	23
4.19	process_block.py	23
4.20	benchmark.py	24
4.21	checkpoint.py	24
4.22	logger.py	25
7.1	Main Execution Flow	32
9.1	Output Zarr Structure	35
11.1	FileNotFoundError	39
11.2	MemoryError	39
11.3	KeyError	39
11.4	ImportError	40
11.5	IndentationError	40
12.1	CITATION.cff	42
B.1	Full config.yaml with Descriptions	45
C.1	Basic Execution	47

C.2	Resuming Execution	47
C.3	Starting Fresh	47
C.4	Analyzing Results	48
C.5	Custom Configuration Example	48

Chapter 1

Introduction

1.1 Overview

ClimateProcessingEngine is an open-source Python framework designed for processing large-scale climate datasets, performing statistical distribution fitting, and generating climatological summaries. The engine is optimized for station-wise time series data with support for multiple variables (temperature, precipitation, etc.) and handles missing data through robust window-based approaches.

1.1.1 Key Features

- **Block-oriented processing** – Handles millions of stations efficiently using configurable block sizes
- **Multi-distribution fitting** – Supports Normal, Skew-Normal, Bimodal, and Pearson Type III distributions
- **Parallel processing** – Optional multi-core execution for accelerated computation
- **Checkpoint recovery** – Automatic resumption from the last completed block
- **Zarr output** – Efficient, chunked, cloud-optimized storage format
- **Comprehensive validation** – Data integrity checks at multiple pipeline stages
- **Benchmarking** – Automatic selection of optimal block size based on system resources

1.1.2 Target Audience

This framework is designed for:

- Climate scientists and meteorologists
- Environmental researchers
- Data engineers working with geospatial time series
- Graduate students in climate science

1.1.3 System Requirements

Table 1.1: Minimum System Requirements

Component	Requirement
Operating System	Windows 10/11, Linux, or macOS
Python	3.8 or higher
RAM	16 GB minimum (32 GB recommended)
Storage	100 GB free space (for intermediate and output data)
CPU	4 cores minimum (8+ recommended for parallel mode)

1.1.4 Dependencies

The following Python packages are required:

```

1 numpy>=1.24.0
2 pandas>=2.0.0
3 xarray>=2023.0.0
4 dask>=2023.0.0
5 rasterio>=1.3.0
6 geopandas>=0.14.0
7 cartopy>=0.22.0
8 matplotlib>=3.7.0
9 scipy>=1.10.0
10 scikit-learn>=1.3.0
11 shapely>=2.0.0
12 netcdf4>=1.6.0
13 zarr>=2.15.0
14 numba>=0.57.0
15 pyyaml>=6.0
16 psutil>=5.9.0 (optional, for RAM monitoring)

```

Listing 1.1: Required Dependencies

Chapter 2

Architecture Overview

2.1 System Architecture

The ClimateProcessingEngine follows a modular, pipeline-based architecture with clear separation of concerns. Figure 2.1 illustrates the overall system design.

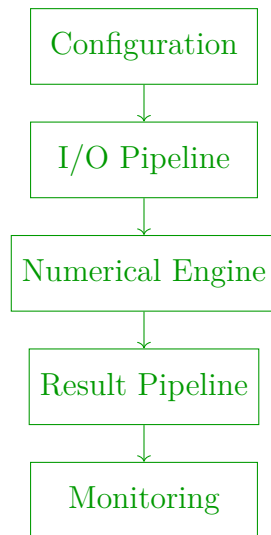


Figure 2.1: System Architecture Overview

2.2 Core Components

2.2.1 Configuration Layer (`constants.py` + `config.yaml`)

The configuration layer centralizes all settings in a single YAML file, which is parsed by `constants.py`. This ensures consistent configuration across all modules.

2.2.2 I/O Pipeline (`io_pipeline/*.py`)

Responsible for reading input data from Zarr archives, assembling blocks, and validating loaded data.

2.2.3 Numerical Engine (numerical_engine/*.py)

Performs the core statistical analysis, including window extraction, distribution fitting, and result aggregation.

2.2.4 Result Pipeline (result_pipeline/*.py)

Validates and writes results to the output Zarr store.

2.2.5 Monitoring (monitoring/*.py)

Handles logging, checkpointing, and benchmarking.

2.2.6 Orchestrator (orchestrator/*.py)

Coordinates the execution flow, managing the block processing loop.

2.3 Data Flow

The data flow follows a linear pipeline:

1. **Load Phase:** Read raw data from input Zarr files for a block of stations
2. **Assembly Phase:** Arrange data into a 4D array (station, year, day, variable)
3. **Validation Phase:** Validate the assembled data against the data contract
4. **Analysis Phase:** For each station, extract windows and fit distributions
5. **Result Validation:** Validate analysis results before writing
6. **Write Phase:** Write validated results to the output Zarr store
7. **Checkpoint:** Record progress for recovery

Chapter 3

Installation and Setup

3.1 Installation

3.1.1 Clone the Repository

```
1 git clone https://github.com/AminFazlKazemi/ClimateProcessingEngine.git
2 cd ClimateProcessingEngine
```

Listing 3.1: Cloning the Repository

3.1.2 Create a Virtual Environment

```
1 # Windows
2 python -m venv venv
3 venv\Scripts\activate
4
5 # Linux/macOS
6 python3 -m venv venv
7 source venv/bin/activate
```

Listing 3.2: Creating Virtual Environment

3.1.3 Install Dependencies

```
1 pip install -r requirements.txt
```

Listing 3.3: Installing Dependencies

3.2 Configuration

3.2.1 Configuration File Structure

The config.yaml file contains all runtime settings:

```
1 # Paths
2 paths:
3   calendar_file: K:/Temp/needed/calendar.txt
4   checkpoint_file: checkpoint.csv
```

```
5  output_dir: I:/climatology_366_rolling
6  output_zarr_name: climatology_stationwise_final.zarr
7  zarr_base: K:/gozareshha/dr vazife/140504 - qc temp/zarr_yearly_monthly
8
9  # Time Range
10 years:
11     start: 1369
12     end: 1399
13 days: 366
14 window_days: 2
15
16 # Processing
17 block_size: 1000
18 cores: 6
19 use_parallel: false
20
21 # Variables
22 variables:
23 - tmin
24 - tmean
25 - tmax
26 fit_variable_index: 1
27
28 # Validation
29 validation:
30     validate_after_load: true
31     validate_before_write: true
32     validate_every_n_blocks: 5
33
34 # Logging
35 logging:
36     level: INFO
37     log_file: climatology.log
38     log_timestamps: true
39
40 # Benchmark
41 benchmark:
42     enabled: true
43     run_on_first_block: true
44     test_block_sizes:
45     - 500
46     - 1000
47     - 1500
48     - 2000
49     - 3000
```

Listing 3.4: Example config.yaml

3.2.2 Understanding Configuration Parameters

Paths

Table 3.1: Path Configuration Parameters

Parameter	Description
<code>calendar_file</code>	Path to the Persian calendar mapping file
<code>checkpoint_file</code>	Location for storing recovery checkpoint
<code>output_dir</code>	Directory where output Zarr will be saved
<code>output_zarr_name</code>	Name of the output Zarr file
<code>zarr_base</code>	Directory containing input Zarr files (organized as YYYY_MM.zarr)

Time Configuration

Table 3.2: Time Configuration Parameters

Parameter	Description
<code>years.start</code>	First year of the analysis (Persian calendar)
<code>years.end</code>	Last year of the analysis (inclusive)
<code>days</code>	Number of days in the year (365 or 366)
<code>window_days</code>	Days before/after each target day for window extraction

The window size is calculated as: `WINDOW_SIZE = 2 * window_days + 1`

Processing Configuration

Table 3.3: Processing Configuration Parameters

Parameter	Description
<code>block_size</code>	Number of stations per processing block
<code>cores</code>	Number of CPU cores to use when parallel processing is enabled
<code>use_parallel</code>	If true, enables multi-core processing

3.3 Preparing Input Data

3.3.1 Input Zarr Structure

The engine expects input Zarr files organized by year and month:

```
1 zarr_base/  
2   1369_01.zarr/  
3     tmin (2D: day, point)  
4     tmax (2D: day, point)  
5     tmean (2D: day, point)  
6     stationid (1D: point)  
7     lat (1D: point)  
8     lon (1D: point)  
9     elev (1D: point)  
10  1369_02.zarr/  
11  ...
```

Listing 3.5: Input Zarr Directory Structure

3.3.2 Calendar File Format

The calendar file (`calendar.txt`) maps Persian dates to day-of-year indices:

```
1 ShamsiDate   ShamsiDOY   GregorianDate  GregorianDOY  
2 13690101     1           19900322      80  
3 13690102     2           19900323      81  
4 ...
```

Listing 3.6: `calendar.txt` Format

Chapter 4

Core Modules Reference

4.1 Configuration Module (constants.py)

4.1.1 Overview

The `constants.py` module loads configuration from `config.yaml` and defines all constants used throughout the engine.

4.1.2 Key Constants

```
1 # Time
2 YEAR_START = 1369
3 YEAR_END = 1399
4 N_YEARS = 31
5 N_DAYS = 366
6 WINDOW_DAYS = 2
7 WINDOW_SIZE = 5                # 2*window_days + 1
8
9 # Variables
10 VARS = ['tmin', 'tmean', 'tmax']
11 N_VARS = 3
12 FIT_VARS = ['tmin', 'tmean', 'tmax']
13 N_OUTPUTS = 87                 # 29 outputs per variable x 3 variables
14
15 # Paths
16 ZARR_BASE = "..."
17 OUTPUT_DIR = "..."
18 OUTPUT_ZARR = "..."
19
20 # Processing
21 BLOCK_SIZE = 1000
22 N_CORES = 6
23 USE_PARALLEL = False
24
25 # Validation thresholds
26 MAX_VALUES_PER_FIT = 155      # N_YEARS * WINDOW_SIZE
27 MIN_VALID_VALUES = 5
28 VALID_BEST_DIST = {-1, 0, 1, 2, 3}
29
30 # Data types
31 FLOAT_DTYPE = np.float32
```

```
32 INT_DTYPE = np.int32
```

Listing 4.1: Constants Defined

4.1.3 Configuration Functions

```
1 def load_config(path):
2     """Load YAML configuration file"""
3     with open(path, 'r', encoding='utf-8') as f:
4         return yaml.safe_load(f)
5
6 def ensure_directories():
7     """Create output directories if they don't exist"""
8     os.makedirs(OUTPUT_DIR, exist_ok=True)
9
10 def validate_config():
11     """Validate configuration values and raise errors on invalid settings
12     """
13     # Checks for valid year range, block size, etc.
```

Listing 4.2: Configuration Functions

4.2 Zarr Schema Module (zarr_schema.py)

4.2.1 Overview

The `zarr_schema.py` module defines the output schema for the Zarr store. It serves as the single source of truth for all output variables.

4.2.2 Variable Definitions

Each output variable is defined as a tuple: (name, dtype, fill_value, description).

Table 4.1: Output Variables per Temperature Variable

Category	Variables	Count
Best Distribution	best_dist	1
Statistics	mean, std, skewness, median, count	5
Normal Parameters	normal_p1, p2, loglik, aicc, bic	5
Skew-Normal Parameters	skew_p1, p2, p3, loglik, aicc, bic	6
Bimodal Parameters	bimodal_p1, p2, p3, p4, p5, loglik, aicc, bic	8
Pearson III Parameters	pearson_p1, p2, p3, loglik, aicc	5
Total per variable		30
Total for 3 variables		90

4.2.3 Key Functions

```

1 def create_zarr_store(output_path, n_stations, chunk_size=(366, 100)):
2     """
3     Create a Zarr store with all output variables.
4
5     Parameters:
6         output_path: str - Path to create the Zarr store
7         n_stations: int - Total number of stations
8         chunk_size: tuple - Chunk size for Zarr arrays (day, station)
9
10    Returns:
11        zarr.Group: The root group of the Zarr store
12    """
13    # Creates arrays for each variable with specified dtype and fill value
14
15 def add_coords_and_metadata(ds, station_ids, lons, lats, elevs):
16     """
17     Add coordinate variables and metadata to an xarray Dataset.
18
19     Parameters:
20         ds: xarray.Dataset - Dataset to augment
21         station_ids: array - Station identifiers
22         lons, lats, elevs: arrays - Geographic metadata
23
24     Returns:
25         xarray.Dataset: Dataset with coordinates and metadata
26     """
27
28 def create_empty_block_result(block_size):
29     """
30     Create an empty block_result dictionary.
31
32     Parameters:
33         block_size: int - Number of stations in the block
34
35     Returns:
36         dict: Empty result container with all variables initialized
37     """

```

Listing 4.3: Zarr Schema Functions

4.3 Calendar Tables Module (calendar_tables.py)

4.3.1 Overview

This module provides calendar conversion functions for the Persian calendar system.

4.3.2 Key Functions

```

1 def is_persian_leap_year(year):
2     """
3     Check if a Persian year is a leap year.
4
5     Parameters:
6         year: int - Persian year
7

```

```

8     Returns:
9         bool: True if leap year
10    """
11
12 def get_persian_month_days(month, year):
13     """
14     Get the number of days in a Persian month.
15
16     Parameters:
17         month: int - Month number (1-12)
18         year: int - Persian year
19
20     Returns:
21         int: Number of days in the month
22     """
23
24 def get_shamsi_doy(year, month, day):
25     """
26     Calculate the day-of-year for a Persian date.
27
28     Parameters:
29         year, month, day: int - Persian date
30
31     Returns:
32         int: Day-of-year (1-based)
33     """
34
35 def build_doy_table():
36     """
37     Build the day-of-year lookup table.
38
39     Returns:
40         ndarray: Shape (N_YEARS, 13, 32) with doy values or -1 for invalid
41         dates
42     """
43
44 def build_calendar_tables():
45     """
46     Build all calendar tables, using calendar.txt if available.
47
48     Returns:
49         dict: Contains doy_table, year_index, year_list, and source
50     """

```

Listing 4.4: Calendar Functions

4.4 Runtime Tables Module (runtime_tables.py)

4.4.1 Overview

This module builds tables needed at runtime, independent of the calendar system.

4.4.2 Key Functions

```

1 def build_window_table():

```

```

2     """
3     Build the window table for all days.
4
5     Returns:
6         list: 366 entries, each containing indices of days in the window
7     """
8     # For each day i, returns [i-2, i-1, i, i+1, i+2] with wrapping at
9     boundaries
10
11 def build_file_map(zarr_base):
12     """
13     Build a map from (year, month) to Zarr file path.
14
15     Parameters:
16         zarr_base: str - Base directory containing Zarr files
17
18     Returns:
19         dict: {(year, month): file_path}
20     """
21
22 def build_runtime_tables(zarr_base):
23     """
24     Build all runtime tables.
25
26     Returns:
27         dict: Contains window_table, file_map, and variable_table
28     """

```

Listing 4.5: Runtime Tables Functions

4.5 I/O Pipeline Module (io_pipeline/*.py)

4.5.1 read_month_files.py

```

1 def read_month_files(block_start, block_size, file_map, year_list):
2     """
3     Read data from Zarr files for a block of stations.
4
5     Parameters:
6         block_start: int - Starting station index
7         block_size: int - Number of stations to read
8         file_map: dict - Mapping from (year, month) to file path
9         year_list: list - All years in the analysis period
10
11     Returns:
12         dict: {(year_idx, month, var_idx): ndarray}
13     """
14     # For each year and month, opens the corresponding Zarr file
15     # Slices the required station range
16     # Extracts temperature variables and applies scaling for tmean
17     # Returns a dictionary of raw arrays

```

Listing 4.6: read_month_files.py

4.5.2 assemble_block.py

```

1 def assemble_block(data_dict, doy_table, block_size, year_list):
2     """
3     Assemble raw data into a 4D array.
4
5     Parameters:
6         data_dict: dict - Raw data from read_month_files
7         doy_table: ndarray - Day-of-year lookup table
8         block_size: int - Number of stations
9         year_list: list - All years
10
11     Returns:
12         ndarray: Shape (block_size, N_YEARS, N_DAYS, N_VARS)
13     """
14     # Initializes array with NaN values
15     # For each year, month, and variable:
16     #     Maps days to correct day-of-year positions
17     #     Copies data to the assembled array
18     # Missing days remain as NaN

```

Listing 4.7: assemble_block.py

4.5.3 validate_block.py

```

1 def validate_block(block_data, block_start, block_size, strict=True):
2     """
3     Validate assembled block data against the data contract.
4
5     Parameters:
6         block_data: ndarray - The assembled block
7         block_start: int - Starting station index
8         block_size: int - Number of stations
9         strict: bool - Whether to raise exceptions on validation failure
10
11     Returns:
12         dict: Validation report with status, warnings, and errors
13     """
14     # Checks:
15     #     - Shape correctness
16     #     - Data type
17     #     - Contiguous memory layout
18     #     - NaN ratio
19     #     - Infinite values
20     #     - Per-station data completeness

```

Listing 4.8: validate_block.py

4.6 Numerical Engine Module (numerical_engine/*.py)

4.6.1 window_engine.py

```

1 def extract_window_values_fast(station_data, window_table, var_idx):
2     """

```



```

3     Extract window values for all days of a station.
4
5     Parameters:
6         station_data: ndarray - Shape (N_YEARS, N_DAYS, N_VARS)
7         window_table: list - Window indices for each day
8         var_idx: int - Variable index to extract
9
10    Returns:
11        list: 366 entries, each containing clean values array or None
12    """
13    # For each day:
14    #     Extracts values from window days
15    #     Removes NaN values
16    #     Returns array if length >= MIN_VALID_VALUES, else None

```

Listing 4.9: window_engine.py

4.6.2 distributions.py

This module implements the core distribution fitting functions using Numba JIT compilation.

Normal Distribution

```

1 @njit
2 def fit_normal_full_numba(data):
3     """
4     Fit a Normal distribution to data.
5
6     Parameters:
7         data: ndarray - Input data
8
9     Returns:
10        tuple: (mean, std, loglikelihood, AICc, BIC, n_parameters)
11    """
12    # Calculates mean and standard deviation
13    # Computes log-likelihood
14    # Calculates AICc and BIC

```

Listing 4.10: Normal Distribution Fitting

Skew-Normal Distribution

```

1 @njit
2 def skewness_to_alpha(skew):
3     """Convert sample skewness to skew-normal shape parameter alpha"""
4     # Maps skewness to alpha using empirical relationship
5
6 @njit
7 def fit_skewnorm_full_numba(data):
8     """
9     Fit a Skew-Normal distribution using method of moments.
10
11    Returns:
12        tuple: (alpha, loc, scale, loglikelihood, AICc, BIC, n_parameters)

```

```

13     """
14     # Estimates initial parameters from moments
15     # Computes log-likelihood
16     # Calculates AICc and BIC

```

Listing 4.11: Skew-Normal Fitting

Bimodal Normal Distribution

```

1 @njit
2 def fit_bimodal_full_numba(data):
3     """
4     Fit a Bimodal Normal distribution using moment-based estimation.
5
6     Returns:
7         tuple: (w1, mu1, sigma1, mu2, sigma2, loglikelihood, AICc, BIC,
8               n_parameters)
9     """
10    # Estimates 5 parameters from moments:
11    #   w1: mixing weight of component 1
12    #   mu1, sigma1: mean and std of component 1
13    #   mu2, sigma2: mean and std of component 2
14    # Uses kurtosis to estimate separation between modes

```

Listing 4.12: Bimodal Fitting

Pearson Type III Distribution

```

1 @njit
2 def fit_pearson3_full_numba(data):
3     """
4     Fit a Pearson Type III (Gamma) distribution.
5
6     Returns:
7         tuple: (shape, scale, location, loglikelihood, AICc, BIC,
8               n_parameters)
9     """
10    # Uses method of moments with skewness adjustment
11    # Applies Wilson-Hilferty transformation for initial estimates
12    # Computes log-likelihood via Gamma PDF

```

Listing 4.13: Pearson Type III Fitting

Master Fitting Function

```

1 @njit
2 def fit_all_distributions_numba(data):
3     """
4     Fit all four distributions and select the best.
5
6     Returns:
7         ndarray: 30-element array containing:
8             [0] best_dist (0=Normal, 1=Skew, 2=Bimodal, 3=Pearson, -1=
9             failed)
10            [1] mean

```

```

10         [2]  std
11         [3]  skewness
12         [4]  median
13         [5]  count
14         [6]  normal_p1 (mean)
15         [7]  normal_p2 (std)
16         [8]  normal_loglik
17         [9]  normal_aicc
18         [10] normal_bic
19         [11] skew_p1 (alpha)
20         [12] skew_p2 (loc)
21         [13] skew_p3 (scale)
22         [14] skew_loglik
23         [15] skew_aicc
24         [16] skew_bic
25         [17] bimodal_p1 (w1)
26         [18] bimodal_p2 (mu1)
27         [19] bimodal_p3 (sigma1)
28         [20] bimodal_p4 (mu2)
29         [21] bimodal_p5 (sigma2)
30         [22] bimodal_loglik
31         [23] bimodal_aicc
32         [24] bimodal_bic
33         [25] pearson_p1 (shape)
34         [26] pearson_p2 (scale)
35         [27] pearson_p3 (location)
36         [28] pearson_loglik
37         [29] pearson_aicc
38     """
39     # Fits all 4 distributions
40     # Selects best using AICc
41     # Returns 30-element array with all parameters

```

Listing 4.14: fit_all_distributions_numba

4.6.3 analyze_station.py

```

1 def analyze_station(station_data, window_table, var_indices):
2     """
3     Analyze a single station for multiple variables.
4
5     Parameters:
6         station_data: ndarray - Shape (N_YEARS, N_DAYS, N_VARS)
7         window_table: list - Window indices
8         var_indices: list - Variable indices to analyze
9
10    Returns:
11        dict: Result dictionary with 87 keys (29 per variable)
12    """
13    # For each variable index:
14    #     Extracts window values using extract_window_values_fast
15    #     For each day:
16    #         Fits distributions using fit_distribution
17    #         Populates result dictionary with parameters

```

Listing 4.15: analyze_station.py

4.6.4 merge_results.py

```

1 def merge_station_result(block_result, station_result, local_idx):
2     """
3     Merge a station's results into the block result.
4
5     Parameters:
6         block_result: dict - Accumulated block results
7         station_result: dict - Results for one station
8         local_idx: int - Position in the block
9     """
10
11 def create_and_merge_results(block_data, window_table, var_indices):
12     """
13     Process all stations in a block and merge results.
14
15     Parameters:
16         block_data: ndarray - Shape (block_size, N_YEARS, N_DAYS, N_VARS)
17         window_table: list - Window indices
18         var_indices: list - Variable indices to analyze
19
20     Returns:
21         dict: Complete block result with 87 variables
22     """
23     # Iterates through stations
24     # Calls analyze_station for each
25     # Merges results into block_result

```

Listing 4.16: merge_results.py

4.7 Result Pipeline Module (result_pipeline/*.py)

4.7.1 validate_result.py

```

1 def validate_result(block_result, block_start, block_size, strict=True):
2     """
3     Validate analysis results before writing.
4
5     Parameters:
6         block_result: dict - Results to validate
7         block_start: int - Starting station index
8         block_size: int - Number of stations
9         strict: bool - Whether to raise exceptions
10
11     Returns:
12         dict: Validation report
13     """
14     # Checks:
15     # - Shape of each variable
16     # - Data type matches schema
17     # - best_dist contains only valid codes
18     # - count values in valid range
19     # - No negative standard deviations
20     # - No infinite values in loglik/AICc/BIC

```

Listing 4.17: validate_result.py

4.7.2 write_block.py

```

1 def write_block(root, block_result, block_start, block_end):
2     """
3     Write block results to Zarr store.
4
5     Parameters:
6         root: zarr.Group - Root of the Zarr store
7         block_result: dict - Results for the block
8         block_start: int - Starting station index
9         block_end: int - Ending station index (exclusive)
10    """
11    # For each variable in the schema:
12    #     Writes the block slice to the Zarr array
13
14 def write_block_safe(root, block_result, block_start, block_end, validate=
15     True):
16     """
17     Write block results with optional validation.
18     """
19     # Validates results if requested
20     # Calls write_block on success

```

Listing 4.18: write_block.py

4.8 Orchestrator Module (orchestrator/*.py)

4.8.1 process_block.py

```

1 class FitError(Exception):
2     """Raised when distribution fitting fails"""
3
4 class DataError(Exception):
5     """Raised when data validation fails"""
6
7 class IOError(Exception):
8     """Raised when I/O operations fail"""
9
10 def process_block(block_start, block_end, block_idx, file_map, doy_table,
11     window_table, year_list, root, var_indices,
12     last_checkpoint_station=None):
13     """
14     Process a single block of stations.
15
16     Parameters:
17         block_start, block_end: int - Station range for this block
18         block_idx: int - Block index
19         file_map: dict - File mapping
20         doy_table: ndarray - Day-of-year table
21         window_table: list - Window table
22         year_list: list - Years in analysis
23         root: zarr.Group - Output Zarr root
24         var_indices: list - Variable indices to analyze
25         last_checkpoint_station: int or None - Last processed station

```

```

26 Returns:
27     dict: Processing statistics for the block
28     """
29     # 1. Load data using read_month_files and assemble_block
30     # 2. Validate loaded data
31     # 3. Analyze using create_and_merge_results
32     # 4. Validate results
33     # 5. Write results
34     # 6. Save checkpoint
35     # 7. Return statistics

```

Listing 4.19: process_block.py

4.9 Monitoring Module (monitoring/*.py)

4.9.1 benchmark.py

```

1 def run_benchmark(load_func, analyze_func, write_func, test_sizes=None):
2     """
3     Run benchmark tests to determine optimal block size.
4
5     Parameters:
6         load_func: function - Function to load a block
7         analyze_func: function - Function to analyze a block
8         write_func: function - Function to write results
9         test_sizes: list - Block sizes to test
10
11     Returns:
12         dict: Benchmark results with recommended block size
13     """
14     # For each test size:
15     #     Measures load, analyze, write times
16     #     Monitors RAM usage
17     #     Counts failed fits
18     #     Calculates stations per second
19     # Returns best performing size
20
21 def print_benchmark_report(report):
22     """
23     Pretty-print benchmark results.
24     """

```

Listing 4.20: benchmark.py

4.9.2 checkpoint.py

```

1 def save_checkpoint(block_idx, station_idx, elapsed=None):
2     """
3     Save checkpoint information to file.
4
5     Parameters:
6         block_idx: int - Current block index
7         station_idx: int - Last processed station
8         elapsed: float - Total elapsed time (optional)

```

```
9      """
10      # Saves to CSV format: block, station, timestamp, elapsed, version
11
12  def load_checkpoint():
13      """
14      Load checkpoint information from file.
15
16      Returns:
17          dict or None: Checkpoint data if exists, None otherwise
18      """
19
20  def delete_checkpoint():
21      """
22      Delete the checkpoint file after successful completion.
23      """
```

Listing 4.21: checkpoint.py

4.9.3 logger.py

```
1  def setup_logger():
2      """
3      Set up logging configuration.
4
5      Returns:
6          logging.Logger: Configured logger with file and stream handlers
7      """
8      # Creates log file in output directory
9      # Configures console and file handlers
10     # Sets formatting with timestamps if configured
11
12  logger = setup_logger()
```

Listing 4.22: logger.py

Chapter 5

Data Contract

5.1 Load Phase Contract

5.1.1 Input

- `block_start`: integer – Starting station index
- `block_size`: integer – Number of stations
- `file_map`: dict – (year, month) → file_path
- `year_list`: list – All years in analysis period
- `doy_table`: ndarray – Shape (N_YEARS, 13, 32)

5.1.2 Output

- `block_data`: ndarray – Shape (block_size, N_YEARS, N_DAYS, N_VARS)
- Data type: float32
- Memory layout: C-contiguous
- Missing values: NaN

5.2 Window Phase Contract

5.2.1 Input

- `station_data`: ndarray – Shape (N_YEARS, N_DAYS, N_VARS)
- `window_table`: list – 366 entries

5.2.2 Output

- List of 366 entries
- Each entry: ndarray or None

- Non-None arrays contain valid values (NaNs removed)
- $\text{Length} \geq \text{MIN_VALID_VALUES} = 5$

5.3 Analyze Phase Contract

5.3.1 Input

- `station_data`: ndarray – Shape (N_YEARS, N_DAYS, N_VARS)

5.3.2 Output

- Dict with 87 keys
- Each key: ndarray – Shape (N_DAYS,)
- Types: `int32` for `best_dist` and `count`
- Types: `float32` for all others

5.4 Write Phase Contract

5.4.1 Input

- `block_result`: dict – 87 variables
- Each variable: Shape (N_DAYS, `block_size`)

5.4.2 Output

- Zarr store with 87 variables
- Shape: (N_DAYS, `n_stations`)
- Coordinated with `day_of_year` and `point` dimensions

5.5 Constants

Table 5.1: Key Constants

Constant	Value	Description
WINDOW_SIZE	5	Days in window (2 before + target + 2 after)
MAX_VALUES_PER_FIT	155	N_YEARS * WINDOW_SIZE
MIN_VALID_VALUES	5	Minimum valid values for fitting
N_OUTPUTS	87	Number of output variables
VALID_BEST_DIST	{-1,0,1,2,3}	Valid distribution codes

5.6 Error Types

Table 5.2: Error Types

Error Type	Description
<code>IOError</code>	Read/write errors (file not found, permission denied)
<code>DataError</code>	Invalid data (shape mismatch, missing values, infinite values)
<code>FitError</code>	Distribution fitting failed (insufficient data, numerical issues)

Chapter 6

Mathematical Background

6.1 Distributions

6.1.1 Normal Distribution

The Normal (Gaussian) distribution has probability density function (PDF):

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Parameters:

- μ : Mean (location)
- σ : Standard deviation (scale, $\sigma > 0$)

6.1.2 Skew-Normal Distribution

The Skew-Normal distribution extends the Normal with a shape parameter for asymmetry:

$$f(x) = \frac{2}{\omega} \phi\left(\frac{x-\xi}{\omega}\right) \Phi\left(\alpha \frac{x-\xi}{\omega}\right)$$

where:

- ϕ : Standard Normal PDF
- Φ : Standard Normal CDF
- ξ : Location parameter
- ω : Scale parameter ($\omega > 0$)
- α : Shape parameter (determines skewness)

6.1.3 Bimodal Normal Distribution

The Bimodal Normal is a mixture of two Gaussian distributions:

$$f(x) = w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) + w_2 \mathcal{N}(x; \mu_2, \sigma_2^2)$$

where:

- w_1, w_2 : Mixing weights ($w_1 + w_2 = 1, 0 \leq w_i \leq 1$)
- μ_1, μ_2 : Component means
- σ_1, σ_2 : Component standard deviations ($\sigma_i > 0$)

6.1.4 Pearson Type III Distribution

The Pearson Type III is a three-parameter Gamma distribution:

$$f(x) = \frac{1}{\Gamma(\alpha)\beta^\alpha} (x - \gamma)^{\alpha-1} \exp\left(-\frac{x - \gamma}{\beta}\right)$$

where:

- α : Shape parameter ($\alpha > 0$)
- β : Scale parameter ($\beta > 0$)
- γ : Location parameter

6.2 Model Selection Criteria

6.2.1 AIC (Akaike Information Criterion)

$$\text{AIC} = 2k - 2 \ln(\hat{L})$$

where:

- k : Number of parameters in the model
- $\hat{L}$: Maximum likelihood estimate

6.2.2 AICc (Corrected AIC)

$$\text{AICc} = \text{AIC} + \frac{2k(k+1)}{n-k-1}$$

where n is the sample size.

6.2.3 BIC (Bayesian Information Criterion)

$$\text{BIC} = k \ln(n) - 2 \ln(\hat{L})$$

6.3 Bimodality Metrics

6.3.1 Ashman's D

$$D = \frac{|\mu_2 - \mu_1|}{\sqrt{\frac{\sigma_1^2 + \sigma_2^2}{2}}}$$

6.3.2 Bimodality Coefficient

$$\text{BC} = \frac{\text{skewness}^2 + 1}{\text{kurtosis} + 1}$$

6.3.3 Overlap Coefficient

$$\text{OVL} = \int_{-\infty}^{\infty} \min\{f_1(x), f_2(x)\} dx$$

6.4 Window-Based Estimation

For each day d , the engine extracts values from days $d - 2$ to $d + 2$ (total 5 days) across all years:

$$\text{Window}_d = \{x_{y,t} : y \in \text{years}, t \in [d - 2, d + 2]\}$$

Where t wraps around at the boundaries (circular year).

Chapter 7

Execution Flow

7.1 Main Function

```
1 def main():
2     # 1. Load configuration
3     # 2. Get station information from input data
4     # 3. Build calendar and runtime tables
5     # 4. Run benchmark (if enabled)
6     # 5. Create Zarr output store
7     # 6. Load checkpoint (if exists)
8     # 7. Process blocks in a loop
9     #     - For each block: load, validate, analyze, write, checkpoint
10    # 8. Finalize Zarr store
11    #     - Add coordinates and metadata
12    #     - Consolidate metadata
13    # 9. Delete checkpoint
14    # 10. Log completion
```

Listing 7.1: Main Execution Flow

7.2 Block Processing Flow

1. Load data for block
2. Validate loaded data
3. Analyze each station
4. Validate results
5. Write results to Zarr
6. Save checkpoint

Figure 7.1: Block Processing Steps

7.3 Recovery Flow

When a failure occurs, the recovery flow is:

1. On startup, check for checkpoint file
2. If checkpoint exists, load the last successful block
3. Resume processing from the next station in that block
4. Continue with the main processing loop

Chapter 8

Input Data Format

8.1 Input Zarr Structure

Each monthly Zarr file must contain:

Table 8.1: Input Zarr Variables

Variable	Shape	Description
tmin	(days, points)	Minimum temperature (raw)
tmax	(days, points)	Maximum temperature (raw)
tmean	(days, points)	Mean temperature (raw, scaled by 10)
stationid	(points,)	Station identifiers
lat	(points,)	Latitude
lon	(points,)	Longitude
elev	(points,)	Elevation

8.2 Data Scaling

The `tmean` data is stored with a scaling factor of 10 (i.e., actual value = stored value / 10). The engine automatically handles this scaling during loading.

8.3 Missing Data

Missing values are stored as `NaN` and are handled appropriately during analysis.

Chapter 9

Output Format

9.1 Zarr Structure

The output Zarr store contains:

```
1 /
2 tmin_best_dist      (day_of_year, point)
3 tmin_mean           (day_of_year, point)
4 tmin_std            (day_of_year, point)
5 tmin_skewness       (day_of_year, point)
6 tmin_median         (day_of_year, point)
7 tmin_count          (day_of_year, point)
8 tmin_normal_p1      (day_of_year, point)
9 tmin_normal_p2      (day_of_year, point)
10 tmin_normal_loglik  (day_of_year, point)
11 tmin_normal_aicc    (day_of_year, point)
12 tmin_normal_bic     (day_of_year, point)
13 tmin_skew_p1        (day_of_year, point)
14 tmin_skew_p2        (day_of_year, point)
15 tmin_skew_p3        (day_of_year, point)
16 tmin_skew_loglik    (day_of_year, point)
17 tmin_skew_aicc      (day_of_year, point)
18 tmin_skew_bic       (day_of_year, point)
19 tmin_bimodal_p1     (day_of_year, point)
20 tmin_bimodal_p2     (day_of_year, point)
21 tmin_bimodal_p3     (day_of_year, point)
22 tmin_bimodal_p4     (day_of_year, point)
23 tmin_bimodal_p5     (day_of_year, point)
24 tmin_bimodal_loglik (day_of_year, point)
25 tmin_bimodal_aicc   (day_of_year, point)
26 tmin_bimodal_bic    (day_of_year, point)
27 tmin_pearson_p1     (day_of_year, point)
28 tmin_pearson_p2     (day_of_year, point)
29 tmin_pearson_p3     (day_of_year, point)
30 tmin_pearson_loglik (day_of_year, point)
31 tmin_pearson_aicc   (day_of_year, point)
32 ... (same for tmean and tmax)
33 day_of_year         (day_of_year,)
34 point               (point,)
35 stationid           (point,)
36 lat                 (point,)
37 lon                 (point,)
38 elev                (point,)
```

39

...

Listing 9.1: Output Zarr Structure

9.2 Metadata

The output Zarr includes the following global attributes:

Table 9.1: Global Attributes

Attribute	Description
description	"Climatology 366 days - Station-wise architecture"
source	Years range, e.g., "Years 1369-1399, Window ± 2 days"
architecture	"Station-wise, Block-oriented"
version	Engine version
n_outputs	Number of output variables
min_valid_values	Minimum values required for fitting
var_defs	List of all variable definitions

Chapter 10

Performance Optimization

10.1 Block Size Selection

The optimal block size depends on:

1. Available RAM
2. CPU cache size
3. Disk I/O speed
4. Number of cores

The benchmarking system automatically evaluates block sizes from 500 to 3000 and selects the one with the highest stations-per-second rate.

10.2 Parallel Processing

When `use_parallel` is enabled:

- The engine uses `multiprocessing` or `concurrent.futures`
- Each worker processes a separate block
- The `cores` parameter controls the number of workers

10.3 Memory Management

1. **Block-based processing:** Data is processed in blocks to limit memory usage
2. **Garbage collection:** Active memory is reclaimed after each block
3. **NaN handling:** Missing values use `float32` to reduce memory footprint
4. **Chunked Zarr:** Output is stored in chunks for efficient access

10.4 Performance Tips

1. Use SSD storage for both input and output data
2. Increase the number of cores if available
3. Use smaller block sizes if RAM is limited
4. Disable validation for production runs to reduce overhead
5. Run on a dedicated machine without memory-intensive applications

Chapter 11

Troubleshooting

11.1 Common Errors and Solutions

11.1.1 File Not Found

```
1  
2 FileNotFoundError: [Errno 2] No such file or directory: 'path/to/file'
```

Listing 11.1: FileNotFoundError

Solution:

1. Verify the file path in `config.yaml`
2. Check that the file exists and is readable
3. Use absolute paths to avoid ambiguity

11.1.2 Memory Error

```
1  
2 MemoryError: Unable to allocate array
```

Listing 11.2: MemoryError

Solution:

1. Reduce the `block_size` in `config.yaml`
2. Disable validation for lower memory usage
3. Use `use_parallel: false`
4. Increase system RAM or use a machine with more memory

11.1.3 KeyError

```
1  
2 KeyError: 'variable_name'
```

Listing 11.3: KeyError

Solution:

1. Check that the variable exists in the input Zarr files
2. Verify variable naming matches the schema
3. Ensure the VARS list in `config.yaml` is correct

11.1.4 ImportError

```
1  
2 ImportError: No module named 'module_name'
```

Listing 11.4: ImportError

Solution:

1. Install missing dependencies
2. Check Python environment (use `which python`)
3. Use `pip install -r requirements.txt`

11.1.5 IndentationError

```
1  
2 IndentationError: unexpected indent
```

Listing 11.5: IndentationError

Solution:

1. Check that Python files use consistent indentation (4 spaces)
2. Avoid mixing tabs and spaces
3. Use an editor with Python indentation support

11.2 Logging

The engine logs all major events to:

- **Console:** Real-time progress information
- **Log file:** `climatology.log` in the output directory

11.2.1 Log Levels

- **DEBUG:** Detailed diagnostic information
- **INFO:** Standard progress information (default)
- **WARNING:** Non-critical issues
- **ERROR:** Critical failures

11.3 Debugging Tips

1. Set `logging.level: DEBUG` for more verbose output
2. Check the log file for detailed error traces
3. Run a small test with `block_size: 100` to verify setup
4. Use Python's interactive debugger (`pdb`) for deep troubleshooting
5. Verify input data quality before large-scale processing

Chapter 12

Support and Community

12.1 Citation

If you use ClimateProcessingEngine in your research, please cite:

```
1 cff-version: 1.2.0
2 title: ClimateProcessingEngine
3 authors:
4   - family-names: Kazemi
5     given-names: Amin Fazl
6 version: v1.0.0
7 date-released: 2026-07-24
8 repository-code: https://github.com/AminFazlKazemi/ClimateProcessingEngine
9 license: MIT
```

Listing 12.1: CITATION.cff

12.2 Repository

The source code is available at:

<https://github.com/AminFazlKazemi/ClimateProcessingEngine>

12.3 License

This project is licensed under the MIT License.

12.4 Contact

For questions, issues, or contributions:

- Email: amin.fazl.kazemi@example.com
- GitHub Issues: <https://github.com/AminFazlKazemi/ClimateProcessingEngine/issues>

Appendix A

Full Variable Definitions

Table A.1: Complete Variable Definitions

Variable Name	Dtype	Description
tmin Variables		
tmin_best_dist	int32	Best distribution code (-1=failed, 0=Normal, 1=Skew, 2=Bimodal, 3=Pearson)
tmin_mean	float32	Mean temperature
tmin_std	float32	Standard deviation
tmin_skewness	float32	Skewness
tmin_median	float32	Median
tmin_count	int32	Number of valid values
tmin_normal_p1	float32	Normal: mean
tmin_normal_p2	float32	Normal: standard deviation
tmin_normal_loglik	float32	Normal: log-likelihood
tmin_normal_aicc	float32	Normal: AICc
tmin_normal_bic	float32	Normal: BIC
tmin_skew_p1	float32	Skew-Normal: alpha (shape)
tmin_skew_p2	float32	Skew-Normal: location
tmin_skew_p3	float32	Skew-Normal: scale
tmin_skew_loglik	float32	Skew-Normal: log-likelihood
tmin_skew_aicc	float32	Skew-Normal: AICc
tmin_skew_bic	float32	Skew-Normal: BIC
tmin_bimodal_p1	float32	Bimodal: w1 (mixing weight)
tmin_bimodal_p2	float32	Bimodal: mu1 (mean of component 1)
tmin_bimodal_p3	float32	Bimodal: sigma1 (std of component 1)
tmin_bimodal_p4	float32	Bimodal: mu2 (mean of component 2)
tmin_bimodal_p5	float32	Bimodal: sigma2 (std of component 2)
tmin_bimodal_loglik	float32	Bimodal: log-likelihood
tmin_bimodal_aicc	float32	Bimodal: AICc
tmin_bimodal_bic	float32	Bimodal: BIC
tmin_pearson_p1	float32	Pearson Type III: shape
tmin_pearson_p2	float32	Pearson Type III: scale
tmin_pearson_p3	float32	Pearson Type III: location
tmin_pearson_loglik	float32	Pearson Type III: log-likelihood

Variable Name	Dtype	Description
tmin_pearson_aicc	float32	Pearson Type III: AICc
tmean Variables (same structure as tmin)		
<i>... (29 variables with tmean_ prefix)</i>		
tmax Variables (same structure as tmin)		
<i>... (29 variables with tmax_ prefix)</i>		

Appendix B

Configuration File Reference

B.1 Complete config.yaml

```
1 # Paths
2 paths:
3   calendar_file: K:/Temp/needed/calendar.txt      # Path to Persian
4   calendar_file      # Path to Persian
5   checkpoint_file: checkpoint.csv                 # Checkpoint file name
6   output_dir: I:/climatology_366_rolling           # Output directory
7   output_zarr_name: climatology_stationwise_final.zarr # Output file name
8   zarr_base: K:/.../zarr_yearly_monthly           # Input Zarr base
9   directory
10
11 # Time Range
12 years:
13   start: 1369                                     # First year (Persian
14   calendar)                                       # First year (Persian
15   end: 1399                                       # Last year (inclusive)
16   days: 366                                       # Days per year (365 or
17   366)                                           # Days per year (365 or
18   window_days: 2                                 # Window size +/- days
19
20 # Processing
21 block_size: 1000                                 # Stations per block
22 cores: 6                                          # CPU cores for parallel
23 use_parallel: false                             # Enable parallel
24 processing
25
26 # Variables
27 variables:
28   - tmin                                           # Variables to process
29   - tmean
30   - tmax
31   fit_variable_index: 1                          # Index for fitting (0=
32   tmin, 1=tmean, 2=tmax)
33
34 # Validation
35 validation:
36   validate_after_load: true                      # Validate after loading
37   validate_before_write: true                    # Validate before writing
38   validate_every_n_blocks: 5                     # Validate every N blocks
```

```
34 # Logging
35 logging:
36     level: INFO                # Log level (DEBUG, INFO,
37     WARNING, ERROR)           # Log file name
38     log_file: climatology.log  # Include timestamps in
39     log_timestamps: true       logs
40 # Benchmark
41 benchmark:
42     enabled: true              # Enable benchmarking
43     run_on_first_block: true   # Run benchmark on first
44     run                        # Block sizes to test
45     test_block_sizes:
46     - 500
47     - 1000
48     - 1500
49     - 2000
49     - 3000
```

Listing B.1: Full config.yaml with Descriptions

Appendix C

Example Usage

C.1 Running the Engine

```
1 # Navigate to the engine directory
2 cd K:/gozareshha/dr vazife/140504 - qc temp/climatology/climatology_engine
3
4 # Run the engine
5 python main.py
```

Listing C.1: Basic Execution

C.2 Resuming After Interruption

If the engine is interrupted, simply run it again:

```
1 python main.py
```

Listing C.2: Resuming Execution

The engine will automatically detect the checkpoint and resume from the last completed block.

C.3 Starting Fresh

To start from scratch:

```
1 # Delete the checkpoint file
2 rm checkpoint.csv
3
4 # Delete the output Zarr (if desired)
5 rm -rf I:/climatology_366_rolling/climatology_stationwise_final.zarr
6
7 # Run the engine
8 python main.py
```

Listing C.3: Starting Fresh

C.4 Analyzing Results

After the engine completes, run the analysis script:

```
1 cd ../../
2 python analyze_distributions1.py
```

Listing C.4: Analyzing Results

C.5 Example: Custom Configuration

```
1 # config.yaml
2 paths:
3     output_dir: /data/climate/output
4     output_zarr_name: my_climate_results.zarr
5     zarr_base: /data/climate/input
6
7 years:
8     start: 1370
9     end: 1400
10
11 block_size: 2000
12 cores: 8
13 use_parallel: true
14
15 variables:
16 - tmin
17 - tmax
```

Listing C.5: Custom Configuration Example

Appendix D

Glossary

Table D.1: Glossary of Terms

Term	Definition
AIC	Akaike Information Criterion – A measure of model quality that balances fit and complexity.
AICc	Corrected AIC – AIC with a correction for small sample sizes.
BIC	Bayesian Information Criterion – An alternative to AIC with a stronger penalty for complexity.
Block	A contiguous group of stations processed together for efficiency.
Checkpoint	A saved state that allows resuming after interruption.
Doy	Day of Year – A number from 1 to 365 (or 366) representing the day.
Fit	The process of estimating distribution parameters from data.
GEV	Generalized Extreme Value – A family of distributions used for extreme events.
Log-likelihood	The logarithm of the likelihood function, used in model selection.
NaN	Not a Number – Used to represent missing data.
Numba	A Python library for just-in-time compilation that accelerates numerical computations.
Pearson III	A three-parameter Gamma distribution, also known as a shifted Gamma.
Skew-Normal	A distribution that extends the Normal with a shape parameter for asymmetry.
Window	A set of days around a target day used for estimation (e.g., ± 2 days).
Zarr	A Python library for chunked, compressed, N-dimensional arrays.
Zarr Store	A directory containing Zarr data and metadata.